In the above code, we called a static function (*compare*) of *QString*, whose third argument is an *enum* "*Qt::CaseInsensitive*". One important thing to remember is the difference between a NULL string and an empty string. Look at the following declarations:

```
QString str1;
QString str("");
```

In the first case, *str1* is created with the default constructor, and is both null and empty. A null string is created when the *QString* object is created with the default constructor. In the second case, we have created the object with a string (""), so Qt will treat it as an empty string, but not a null string.

Working with lists

After the basic data classes, let's work on container classes. To begin with, we have *QList*, a generic container class. Specify the type of item to hold in a *QList* with the general form:

```
QList<type> mylist;
```

Use *int*, *QString*, *Qdate*, etc, as the *'type'*. Let's try some sample code:

```
#include <QtCore/QCoreApplication>
#include<QtCore>
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);
    QList<QString> mylist;
    mylist << "Two" << "Three";
    mylist.prepend("One");
    mylist.append("Four");
    for(int i=0; i< mylist.size(); i++)
    {
        qDebug() << mylist.at(i);
    }
    while(!mylist.isEmpty())
        mylist.takeFirst();
    qDebug() << "The list size = " << mylist.size();
    return a.exec();
}
```

The output of the above code is as follows:

```
"One"
"Two"
"Three"
"Four"
The list size =  0
```

In the above code, we used '<<' to insert items in *mylist*. Member functions let us prepend and append items to the list. To access list elements, we use its index, with the member function *'at(index)'*. To remove items from the head, we use *'takeFirst()'*; and to remove items from the tail, we use *'takeLast()'*.

Iterator class

Instead of using an index, we can use iterators. The Qt iterator class is *QListIterator*. Let's replace the *for* loop code in the above example with the following code:

```
QListIterator<QString> it(mylist);
while(it.hasNext())
    qDebug() << it.next();
```

Here, we created a *QListIterator* object to iterate over a *QString* list. We passed the *QList* object *mylist* to the constructor of *QListIterator*. This positions the iterator at the first element of *mylist*. The *while* loop keeps fetching and outputting items until the iterator reaches the end of the list.

The *QList* class provides STL (Standard Template Library) based iterators too. In terms of execution, they are faster (slightly) than the method described above. Here is a sample of code:

```
QList<QString>::const_iterator it;
for (it = mylist.constBegin(); it != mylist.constEnd(); it++)
    qDebug() << *it;
```

If you're interested in this, you can find more details on iterators in the Qt documentation.

Let's move on. An important class derived from *QList* handles *QStrings* in a proper manner. The class *QStringList* is a reimplementation of *QList<QString>*. I use this class when I need to split a string into sub-strings. Let's try some sample code:

```
#include <QtCore/QCoreApplication>
#include<QtCore>
int main(int argc, char *argv[])
{
    QCoreApplication a(argc, argv);
    QStringList mylist;
    QString str("Linux Kernel 3.0");
    mylist = str.split(" ");
    for(int i=0; i<mylist.size(); i++)
        qDebug() << mylist.at(i);
    return a.exec();
}
```

Compile and run the program. The output will be as follows:

```
"Linux"
"Kernel"
"3.0"
```

We have created a *QStringList* object, then split the *'str'* string on each occurrence of a space, and assigned the resultant *QStringList* to *mylist*. We can then use indexes or iterators to access items.